

When Does LLM-Serving Scheduler Adaptation Matter? Action Opportunity and Causal Headroom in Production-Derived Replay

Soroush Vahidi

New Jersey Institute of Technology, Newark, NJ, USA

Abstract

Adaptive scheduling for large language model (LLM) serving pays off only if the live system reaches states where a different executable action exists and taking it improves performance. We measure it before any adaptive scheduler is designed. In a deterministic simulator replaying production-derived traces (Azure code, Azure conversation, BurstGPT), we count how often a six-policy portfolio proposes an action different from the single best scheduler’s and, on fresh non-overlapping windows, force one alternative action for one step before returning to the default. Under abundant resources, replay was action-null in about one million decision states. Capping key-value (KV) cache capacity or concurrent sequences created disagreement; scaling arrival rates up to eight times did not. In a preregistered analysis of 720 fresh disagreement states, 590 (81.9%) had positive one-step latency headroom; the state-weighted mean oracle headroom, the best available one-step latency reduction, was 2.00 ms (95% window-clustered confidence interval 0.19–3.55 ms). Post hoc analyses show that the sign is stable across weightings and leave-one-out analyses, but the magnitude is not: the median is 0.20 ms, and one of 36 windows holds 88% of the total headroom. A bounded vLLM probe shows that resource pressure changes scheduler behavior but does not validate the simulated latency effect. Adaptation opportunity is real but sparse, pressure-dependent, and generally modest; detecting it does not show that a deployed adaptive scheduler is worthwhile.

Keywords: LLM serving, request scheduling, performance evaluation, workload replay, resource contention, causal evaluation

1. Introduction

Adaptive scheduling in large language model (LLM) serving is usually motivated by heterogeneous prompts, decode demand, key-value (KV) cache residency, queueing, and deadlines. It can help only if the live system reaches states where a different executable action exists, such as another admission, batch, or request order, and only if taking that action improves the target metric by more than the cost of choosing it. How often this happens is rarely measured before a scheduler is designed.

Common evaluation practice does not answer this. Ranking policies on whole-trace metrics, or reporting the gap to a virtual best solver (VBS) as in algorithm portfolios [1, 2], shows that policies differ but aggregates thousands of decisions; it cannot say whether an alternative was available at a given state. Policies with different scores can issue the same batch, and an alternative that exists may not reduce latency. Recent systems address *how* to adapt [3, 4, 5, 6]. We ask the prior question: *when is there a different useful action to take, and how much is it worth?*

We answer it with a staged decomposition,

$$\begin{aligned} \text{resource pressure} &\longrightarrow P(D) \longrightarrow P(B_{\text{LAT}} \mid D) \\ &\longrightarrow \text{headroom magnitude} \\ &\longrightarrow \text{predictability and deployment.} \end{aligned} \tag{1}$$

Here D means that at least one canonical executable alternative differs from the default single best scheduler (SBS), and B_{LAT} that at least one such alternative lowers one-step mean latency relative to that default. These are distinct quantities: pressure can make policies disagree often without the disagreement being useful, and useful disagreement can be rare or small. The last arrow, whether a selector could exploit the opportunity, is not measured here.

This is a performance-evaluation question: at which operating points does scheduler choice have causal performance consequences? Answering it takes workload characterization, controlled resource interventions, replay-based simulation, and inference that respects the dependence among states from one trace window. Recent work in *Performance Evaluation* treats job assignment for machine-learning inference [7], dispatching policies on Google and Alibaba workloads [8], latency prediction for GPU-accelerated model

serving [9], and adaptive machine-learning training on serverless clouds [10]; our measurement-first view characterizes the region in which such mechanisms have something to choose between.

We replay production-derived traces (Azure 2023 code and conversation [11], BurstGPT [12]) in a deterministic simulator. Faithful replay of 60 windows gives the native baseline; one-axis interventions on active-sequence capacity, KV capacity, and arrival rate (1,080 conditions) locate where disagreement appears. On fresh non-overlapping windows we clone the simulator at each disagreement state, force one alternative action once, and return to the default. The primary endpoint, oracle latency headroom (the largest one-step mean-latency reduction among the alternatives, zero if none helps), was fixed in a preregistered protocol; the robustness analyses of the frozen results are post hoc. A bounded vLLM [13] probe checks the mechanism in a real engine.

Native replay is action-null under abundant resources. Capacity caps on active sequences or KV cache create disagreement; arrival-rate scaling up to eight times does not. Of 720 fresh disagreement states, 590 (81.9%; 589 excluding one floating-point-residue state) had positive headroom, with a state-weighted mean of 2.00 ms (95% window-clustered confidence interval 0.19–3.55 ms). The sign is robust: the mean stays positive under equal-window, equal-regime, and equal-workload weighting and when any single window or regime is removed. The magnitude is not uniform: the median is 0.20 ms, only 22.5% of states offer more than 2 ms, and one of 36 windows carries 88% of the total headroom. The vLLM probe shows that pressure alters queueing and scheduling behavior but does not validate the simulator’s latency magnitudes. Adaptation opportunity is thus real but sparse, pressure-dependent, and generally modest, and detecting it does not show that a deployed scheduler is worthwhile. The evidence covers three trace families, a six-policy portfolio, a simulator, and one bounded probe on a single model and GPU; the oracle is an opportunity bound, not a scheduler.

The study addresses five research questions, ordered along the decomposition.

RQ1 – Native opportunity. How often does the SBS face a different executable action from the six-policy portfolio, in faithful replay of production-derived traces with abundant resources?

RQ2 – Pressure-induced opportunity. Which resource-pressure dimensions create action disagreement, and does the transition reproduce on

untouched windows?

RQ3 – Causal value. At disagreement states, how often does forcing one alternative action for one step reduce latency, and does the answer depend on the objective?

RQ4 – Magnitude and concentration. How large is the headroom, is it typical or concentrated, and how sensitive is it to weighting and to individual windows or regimes?

RQ5 – System correspondence. Does a bounded vLLM probe show the corresponding pressure and scheduling mechanism?

1.1. Contributions

This paper makes five contributions.

1. A staged action-opportunity evaluation that separates executable disagreement, beneficial disagreement, and headroom magnitude from policy-score diversity, with prevalence measured over all valid decision states and uncertainty quantified by trace window.
2. A characterization of where disagreement appears: native replay is action-null, and only capacity limits on active sequences or KV cache, not arrival-rate scaling, expose alternatives; the transition reproduces on untouched windows.
3. A preregistered one-step causal measurement on 720 fresh states, with latency as the primary endpoint because deadline goodput saturates at 1.0 and hides latency differences.
4. A robustness and concentration analysis showing that the sign of the headroom is robust but its magnitude is concentrated, so a mean reported without its distribution and cluster composition overstates the typical opportunity.
5. Bounded system evidence that resource pressure changes queueing and scheduling behavior in vLLM, without validating simulated latency magnitudes or a deployed selector.

Section 2 positions the work, and Section 3 defines the estimands and evidence design. Sections 4–6 report the native, pressure, and fresh causal results, Section 7 tests their robustness and concentration, Sections 8 and 9 cover objective sensitivity and the vLLM probe, and Section 10 interprets the evidence, its practical implications, and its limits before Section 12 concludes.

2. Related Work and Positioning

LLM-serving research spans model execution, prefill/decode scheduling, resource management, routing, caching, and cluster control. Orca and vLLM established iteration-level serving and paged KV management [14, 13]. Sarathi-Serve, DistServe, Splitwise, and FastServe study chunking, phase separation, and preemptive scheduling [15, 5, 16, 17]. Llumnix and SOLA adapt scheduling or state-aware service to dynamic conditions [3, 4]; Libra studies flexible request partitioning [18]; Mooncake and Strata emphasize KV-centric or hierarchical context caching [19, 20]; MorphServe changes model precision and KV capacity under pressure [21]; and DynamoLLM reconfigures cluster resources for performance and energy [22]. BurstGPT and the Azure traces provide workload evidence [12, 11], and a recent longitudinal study by Nixon et al. examines workload evolution, caching, and load balancing over a year of production traffic [23].

ServeGen characterizes and generates production LLM workloads [24], and OpenTela unifies decentralized computing resources for heterogeneous LLM serving [25]. LMetric combines a KV-aware indicator with an instance-load indicator, reports production-style workloads, and derives conditions under which its multiplicative combination can fail [6]. These works address how to build, evaluate, or operate serving mechanisms. The question here precedes them: in a fixed workload and resource regime, do the default and its alternatives expose different executable actions at all, and does forcing one alternative have default-relative causal value?

Recent papers in *Performance Evaluation* address neighboring problems: job assignment in machine-learning inference systems with accuracy constraints [7], dispatching policies in data-center clusters studied on Google and Alibaba workloads [8], resource-aware latency prediction for GPU-accelerated model serving [9], and adaptive machine-learning training on serverless clouds [10]. They concern the design, evaluation, or performance prediction of assignment, dispatching, and resource-management mechanisms. This paper asks whether, in a given workload and resource state, an alternative scheduling action exists and has causal one-step value relative to a strong default, before a mechanism for choosing among actions is designed. Table 1 summarizes the systems closest to this measurement.

The SBS and VBS quantities used below are non-deployable reference points for measuring opportunity, in the tradition of algorithm portfolios and selection [1, 2, 26, 27]. The contribution is their application to executable

Table 1: Systems closest to this study, by focus and evaluation setting, and their relation to the present measurement.

Work	Focus	Workloads and setting	Relation to this study
ServeGen [24]	Workload characterization and generation	Production workloads	Workload realism; no scheduler-action measurement
LMetric [6]	KV- and load-aware request scheduling	Production-style workloads; real system	Mechanism whose decision states this measurement could examine
OpenTela [25]	Control plane for decentralized serving	Operational traces; real system	Operation, not action opportunity
Llumnix, SOLA [3, 4]	Dynamic, state-aware scheduling	Selected load regimes; systems	Adaptation whose opportunity is measured here
Mooncake, Strata [19, 20]	KV-centric context caching	Cache and resource regimes; real systems	Addresses KV capacity, a pressure axis studied here
This study	Existence and one-step causal value of alternative actions	Production-derived replay under controlled interventions; vLLM probe	—

scheduler actions in the staged characterization of Eq. 1.

3. Methods

3.1. Evidence hierarchy and workload inventory

We report four evidence tiers. Tier 1 is native production-derived trace replay. Tier 2 is production-derived aggregate or window-level evaluation. Tier 3 is a controlled transformation of a real trace, such as one-axis resource caps. Tier 4 is synthetic or mechanistic workload construction. RQ1 uses Tier 1; RQ2 uses Tiers 2–3; fresh causal headroom uses a preregistered Tier-3

intervention over untouched windows; the vLLM result is a bounded system correspondence probe.

The faithful corpus contains 20 windows from each of Azure 2023 code, Azure 2023 conversation, and BurstGPT. Source order and token-length/arrival structure are retained. Service-level objective (SLO) and class overlays are deterministic simulator inputs; future actual output lengths are hidden from policies. The fresh confirmatory campaign uses 20 untouched windows per workload family for support mapping, with causal labeling limited to the frozen selected regimes. BurstGPT did not contribute a selected fresh causal disagreement regime.

The historical six-policy portfolio contains full prefill, small-chunk prefill, estimated-service-first (ESTF), weighted fair sharing (WFS), least-laxity first (LLF), and a KV-constrained policy. The six-policy portfolio is fixed for a default-relative characterization; it is not an exhaustive set of current schedulers.

3.2. Canonical action opportunity

Let $\mathcal{A}_6(s)$ be the set of canonical executable actions produced by the six portfolio policies at state s , after action deduplication. A canonical action is the simulator-executable admission, batch, and request ordering decision after infeasible or semantically identical variants have been collapsed. The default scheduler is the SBS, the fixed best single policy used as the canonical baseline for the frozen replay protocol; it is not refit per state. Let $a_{\text{SBS}}(s)$ be its action. We define

$$D(s) = \mathbf{1} [\exists a \in \mathcal{A}_6(s) : a \neq a_{\text{SBS}}(s)]. \quad (2)$$

The opportunity prevalence is $P(D) = \sum_s D(s) / |\mathcal{S}_{\text{all}}|$, where the denominator is every valid SBS decision state, not only states where policy scores differ. A policy ranking change that produces the same admission, batch, and request order is action collapse, not opportunity. An alternative action is any executable member of $\mathcal{A}_6(s)$ that is not equal to $a_{\text{SBS}}(s)$.

This definition matters operationally. Whole-policy comparisons can show a latency or goodput difference accumulated over many decisions, but cannot say whether an alternative existed at a particular default state. Conversely, action diversity alone says nothing about usefulness.

3.3. One-step causal headroom

For every disagreement state s and canonical alternative a , we clone the state, force exactly one action, and return to the SBS for the continuation. Denote the resulting continuation value by $Q_{\text{SBS}}(s, a)$. This is a local counterfactual, not selector performance: it estimates the one-step effect of changing the current action while holding the subsequent default policy fixed.

For the fresh primary latency endpoint, continuation mean latency includes active, queued, and future requests belonging to the continuation, excluding requests completed before s . We define

$$\begin{aligned} A_{\text{LAT}}(s, a) &= L_{\text{SBS}}(s) - L_{\text{CF}}(s, a), \\ H_{\text{LAT}}(s) &= \max \left(0, \max_a A_{\text{LAT}}(s, a) \right), \end{aligned} \tag{3}$$

where positive values reduce mean latency. The beneficial-state indicator is $B_{\text{LAT}}(s) = 1$ iff $\max_a A_{\text{LAT}}(s, a) > 0$. We report $P(B_{\text{LAT}} \mid D)$ and mean H_{LAT} separately from $P(D)$. The conceptual aggregate opportunity is therefore

$$P(D) \times P(B_{\text{LAT}} \mid D) \times \text{effect magnitude}, \tag{4}$$

not an exact deployable-performance equation. A high conditional benefit can coexist with a tiny overall opportunity.

The confirmatory mean uses a faithful-window clustered percentile bootstrap with 2,000 replicates and seed 20260920. The cluster unit is the original fresh trace window, identified by workload source and window index, contributing disagreement states, so multiple states from one window do not become independent experimental units. Its verdict requires positive mean headroom and a clustered 95% lower endpoint above zero.

A pressure condition is a workload window plus one frozen one-axis intervention, such as a KV-capacity cap, active-sequence cap, or arrival-rate scaling factor. “Support” and “opportunity” refer only to the existence of executable non-SBS actions under that condition; usefulness requires the separate causal latency test above.

3.4. Post hoc robustness analyses

The analyses in Section 7 were specified after the fresh outcomes were known. They qualify the preregistered result and do not replace it: the endpoint H_{LAT} , the strict criterion B_{LAT} , the 720 states, and the selected regimes are unchanged. A *window* is a workload source and window index

(36 windows contribute states); a *regime* is a workload, pressure axis, and pressure setting pooled over windows (five selected regimes).

Practical thresholds. For $\epsilon \in \{0.1, 0.25, 0.5, 1, 2, 5\}$ ms we report the number and share of disagreement states with $H_{\text{LAT}}(s) > \epsilon$, that is $P(H_{\text{LAT}} > \epsilon \mid D)$, and the number of windows that contain at least one such state.

Numerical residue. The preregistered criterion is applied unchanged in double precision. Simulated latency differences are integer multiples of the decode step divided by the request population, so no genuine advantage is smaller than about 5.2×10^{-6} s. Only to identify floating-point residue, we also count states whose best advantage exceeds 10^{-12} s.

Aggregation. For a unit u (a window, regime, or workload source) let D_u be its disagreement states and $\bar{H}_u = |D_u|^{-1} \sum_{s \in D_u} H_{\text{LAT}}(s)$. The preregistered estimate is the *state-weighted mean* $\bar{H} = |D|^{-1} \sum_{s \in D} H_{\text{LAT}}(s)$, in which every state has weight $1/|D|$. The *equal-window*, *equal-regime*, and *equal-workload* means are $U^{-1} \sum_u \bar{H}_u$ over the 36 windows, five regimes, or two workloads, so every unit has weight $1/U$ however many states it contains. These are different estimands, not alternative estimates of one quantity; the beneficial share is aggregated in the same way. Intervals for the alternative means use the same window-clustered percentile bootstrap with 200,000 replicates. For the equal-regime and equal-workload means, regimes and workloads are fixed strata and window–regime cells are resampled within each stratum, which ignores dependence between regimes that share a window.

Leave-one-out and concentration. For each window (36) and each regime (5) we recompute the state-weighted mean and beneficial share from the remaining states, with a window-clustered bootstrap interval (200,000 replicates). Concentration is the share of the total summed headroom $\sum_s H_{\text{LAT}}(s)$ contributed by each window and regime, and the effective number of windows $1/\sum_w p_w^2$, where p_w is the share of window w .

Interval diagnostics. To check the preregistered interval we repeat the percentile bootstrap with 10^6 replicates, add a bias-corrected and accelerated (BCa) interval, and compute a delete-one-window jackknife t interval. These are diagnostics, not replacements.

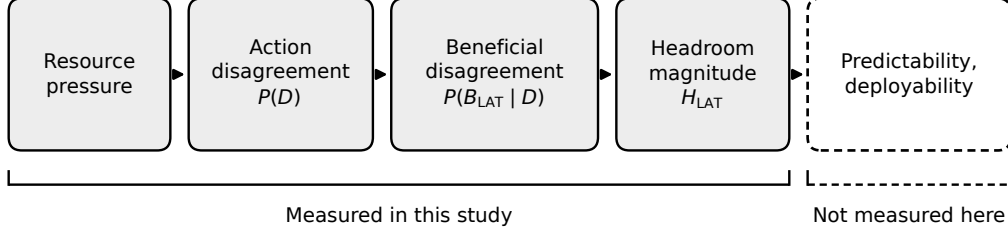


Figure 1: Staged evidence chain (Eq. 1): resource pressure, canonical action disagreement $P(D)$, beneficial disagreement $P(B_{\text{LAT}} | D)$, headroom magnitude H_{LAT} , and predictability and deployability. The study measures the first four stages (solid boxes); the fifth (dashed box) is not measured.

3.5. Objective audit

The historical objective is arrival-normalized weighted goodput (ANWG),

$$\text{ANWG} = \frac{\sum_i w_i \mathbf{1}[i \text{ completes by its SLO}]}{\sum_i w_i}. \quad (5)$$

The earlier terminal arrival-normalized weighted goodput result is retained as an objective audit, not as fresh latency evidence. The latency study uses Eq. 3; its mean-latency result is confirmatory only for the fresh untouched causal universe.

4. Results: Native Action Opportunity

Under the frozen abundant-resource baseline, all three production-derived workloads were canonically action-null (Figure 1 summarizes the evidence chain; Figure 2 gives the rates). Azure code had 0 disagreements in 99,992 SBS decision states; Azure conversation had 0 in 461,985; and BurstGPT had 0 in 440,461. The corresponding maximum KV utilizations were 0.0027, 0.0017, and 0.0038, with no active-cap binding states. Policy-ranking proxy scores differed in a small number of states, but the canonical actions collapsed to the SBS action. In this resource regime the six-policy portfolio offered no executable alternative.

5. Results: Pressure-Induced Disagreement

The pressure map evaluated 1,080 one-axis conditions: 20 windows per workload family, each under settings on the active-sequence, KV-capacity,

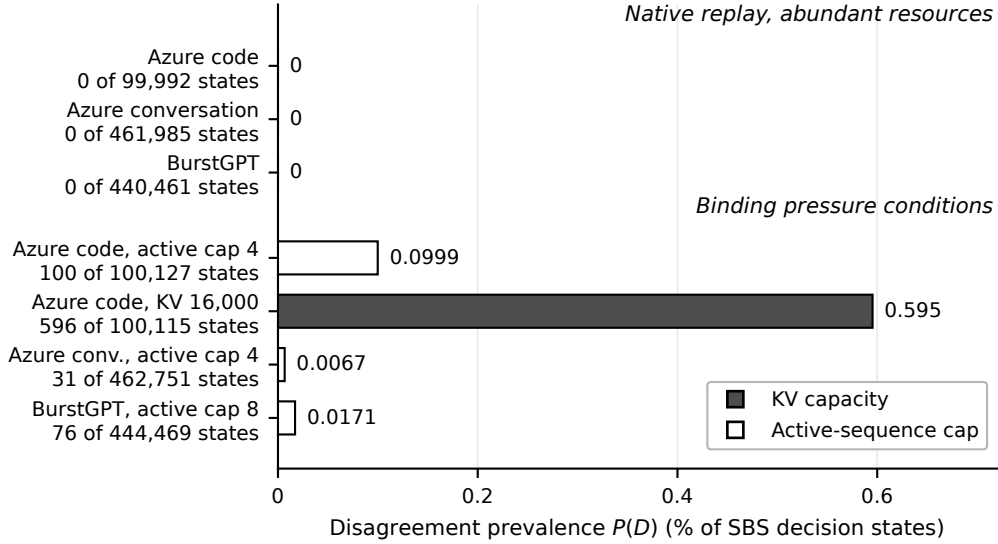


Figure 2: Disagreement prevalence $P(D)$, the percentage of SBS decision states that are disagreement states (counts in the axis labels), for native replay of the three workloads and for selected pressure conditions: Azure code with active-sequence cap 4 and with KV capacity 16,000 tokens, Azure conversation (conv.) with cap 4, and BurstGPT with cap 8. Native replay produced no disagreement state. Fill denotes the pressure axis.

and arrival-rate axes (Figure 3; Table 2). Of these, 926 conditions were valid and unconstrained, 38 partially constrained, 96 strongly constrained, and 20 horizon-invalid. Across valid conditions, 11,328 canonical disagreement states were observed.

Arrival scaling through $8\times$ remained action-null for all three workload families. Active-sequence pressure created disagreement at a cap of 16 (BurstGPT), 8 (Azure code), or 4 (Azure conversation), and KV pressure created it at a capacity of 16,000 tokens in all three workloads (Table 2). Disagreement became sustained, in at least two windows, at caps of 8 (BurstGPT) and 4 (Azure code and conversation) and at KV capacities of 16,000 (Azure code and BurstGPT) and 8,000 (Azure conversation).

The transition (Table 2) is the main regime result: pressure does not automatically create useful diversity, but concrete resource binding can turn a collapsed action set into a nontrivial one. In the studied traces, arrival-rate scaling alone did not create support through the tested range; tight KV capacity and active-sequence limits were the operative dimensions.

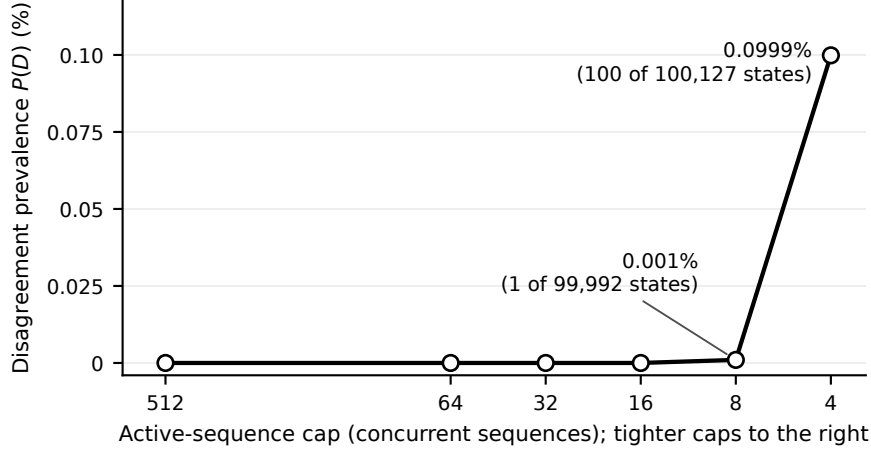


Figure 3: Disagreement prevalence $P(D)$ (percentage of SBS decision states, pooled over the 20 windows) for Azure code as the active-sequence cap is tightened. The horizontal axis is logarithmic and deliberately reversed, so pressure increases from left to right; the six points are the frozen cap settings.

6. Results: Fresh Support and Causal Latency Headroom

The fresh support mapping used untouched windows and the frozen pressure protocol. The causal labeling plan selected five Azure code and conversation regimes (active-sequence and KV-capacity caps) with 720 disagreement states. BurstGPT contributed no selected causal regime.

The confirmatory intervention found 590 beneficial states (Table 3; Figure 4), giving

$$P(B_{\text{LAT}} \mid D) = 590/720 = 0.8194 \text{ (81.9\%)}.$$

Mean oracle latency headroom was 0.001995791 s (1.9958 ms), with a clustered 95% confidence interval (CI) of [0.000190884, 0.003546196] s ([0.1909, 3.5462] ms). The preregistered verdict was positive. The result is statistically supported but operationally modest: it establishes measurable default-relative latency opportunity at disagreement states, whereas whether its magnitude exceeds the engineering cost of adaptation is a separate question. The conditional share is defined on disagreement states only, and because $P(D)$ is often below one percent in the selected regimes it is not the share of all scheduler decisions that could be improved.

Table 2: Pressure settings at which canonical disagreement first appears. Settings are the active-sequence cap (concurrent sequences) and the KV capacity (tokens). “First binding” is the first setting at which the constraint binds; “sustained” is the first setting with disagreement in at least two windows.

Workload	Pressure axis	First binding	First disagreement	Sustained
Azure code	Active-sequence cap	8	8	4
Azure code	KV capacity	16,000	16,000	16,000
Azure conversation	Active-sequence cap	4	4	4
Azure conversation	KV capacity	8,000	16,000	8,000
BurstGPT	Active-sequence cap	16	16	8
BurstGPT	KV capacity	16,000	16,000	16,000

These preregistered values are state-weighted over disagreement states drawn from 36 windows, and they should be read with the post hoc analyses of Section 7: the typical state has a much smaller headroom (median 0.20 ms), and the mean is highly concentrated in one window. One of the 590 beneficial states has an advantage of 5.6×10^{-17} s, exactly one unit in the last place of the 0.357 s latencies compared, with both branches completing the same requests; a post hoc guard of 10^{-12} s gives 589/720 (81.8%) and changes the mean by less than 10^{-19} s.

6.1. Opportunity decomposition

Figure 5 makes the distinction between prevalence and value concrete. Azure-code KV capacity = 16,000 has the largest selected $P(D)$ and the largest mean headroom; active-cap regimes have much smaller $P(D)$ and correspondingly smaller aggregate opportunity even when some disagreement states are beneficial. A practitioner should therefore measure all three factors in Eq. 4: whether a different action exists, whether it helps conditional on existing, and whether its effect is large enough to matter relative to system noise and control overhead.

The practical reading is deliberately staged. Under abundant resources, this study observed no canonical disagreement and therefore no demonstrated action support for adaptive selection. Under constrained KV or active-sequence regimes, disagreement emerges and creates an executable opportunity. Within those disagreement states, alternatives with positive headroom

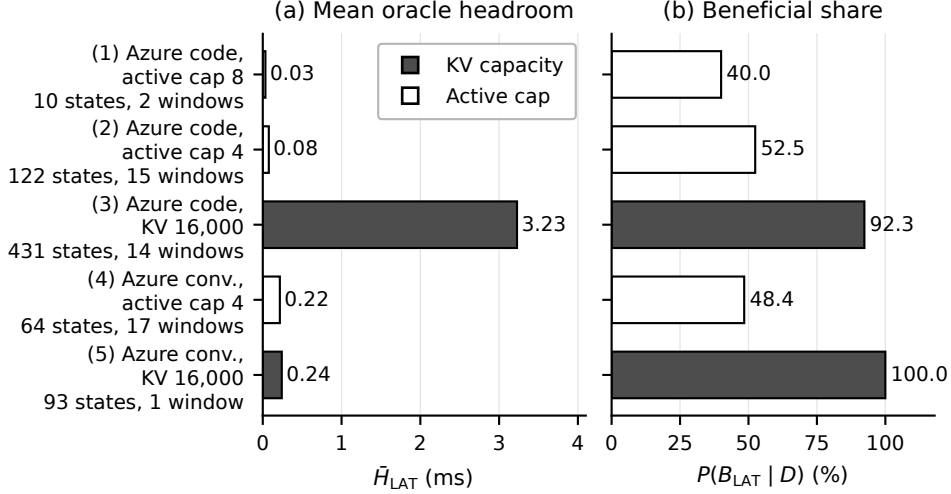


Figure 4: Fresh causal characterization of the five selected regimes, numbered as in Table 3 (conv. = conversation; active cap = active-sequence cap; KV = KV capacity in tokens). (a) State-weighted mean oracle headroom $\bar{H}_{LAT}$ within each regime (ms). (b) Beneficial share $P(B_{LAT} | D)$: the percentage of the regime’s disagreement states with positive headroom under the preregistered criterion. Fill denotes the pressure axis. State and window counts are shown in place of intervals, because window-clustered intervals are undefined for regimes with one or two windows.

are frequent but mostly small (Section 7). Opportunity and headroom still do not prove that a deployed adaptive selector is worthwhile.

7. Results: Robustness and Concentration of Causal Headroom

The preregistered result of Section 6 is a state-weighted mean over 720 disagreement states drawn from 36 windows. This section asks, post hoc, how large the headroom is in a typical state and how far the mean depends on which windows and regimes contribute (Section 3.4). The preregistered criterion and estimate are not modified.

7.1. Effect scale and distribution

Table 4 counts the states whose headroom exceeds each practical threshold. The 81.9% beneficial share is not the share of states that offer material gains: 60.0% of states exceed 0.1 ms, 29.4% exceed 0.5 ms, 22.5% exceed 2 ms, and 17.9% exceed 5 ms. The larger effects are also concentrated.

Table 3: Fresh causal characterization by selected regime, numbered as in Figures 4 and 5; “conv.” is conversation, “active cap” the active-sequence cap, and “KV” the KV capacity in tokens. $P(D)$ is the percentage of valid SBS decision states in the regime’s conditions that are disagreement states, and $P(B_{\text{LAT}} \mid D)$ the percentage of disagreement states with positive oracle headroom (strict criterion). Mean H_{LAT} is the state-weighted mean within the regime. A window is counted in a regime if it has at least one disagreement state there; windows can contribute to several regimes, so counts are not additive, and the 36 distinct workload–window pairs are the bootstrap clusters.

Regime	States	Windows	$P(D)$ (%)	$P(B_{\text{LAT}} \mid D)$ (%)	Mean H_{LAT} (ms)	Headroom share (%)
(1) Azure code, active cap 8	10	2	0.0106	40.0	0.033	0.02
(2) Azure code, active cap 4	122	15	0.129	52.5	0.077	0.65
(3) Azure code, KV 16,000	431	14	0.456	92.3	3.227	96.80
(4) Azure conv., active cap 4	64	17	0.0108	48.4	0.217	0.97
(5) Azure conv., KV 16,000	93	1	0.0157	100.0	0.241	1.56
All selected regimes	720	36	—	81.9	1.996	100.00

Above 0.5 ms, 201 of the 212 states belong to the Azure-code KV-capacity = 16,000 regime, and only 4 of the 36 windows contain a state above 2 ms.

The distribution is strongly right-skewed (Figure 6a). The median headroom is 0.197 ms, about one tenth of the 1.996 ms state-weighted mean, and 77.4% of states lie at or below that mean. The quartiles are 0.023 and 1.19 ms, the 90th and 95th percentiles are 8.4 and 11.3 ms, and the maximum is 12.1 ms. A further 130 states (18.1%) have exactly zero headroom. Among the 589 states with positive headroom the median is 0.29 ms.

7.2. Sensitivity to aggregation and to individual windows and regimes

Every aggregation of Table 5 yields a positive mean whose interval excludes zero, but the magnitude depends on what is weighted: from 1.996 ms (state-weighted) to 0.327 ms (equal-window), a factor of about six. The beneficial share is likewise composition-sensitive, from 0.819 (state-weighted) to 0.632 (equal-window). The state-weighted mean is the preregistered esti-

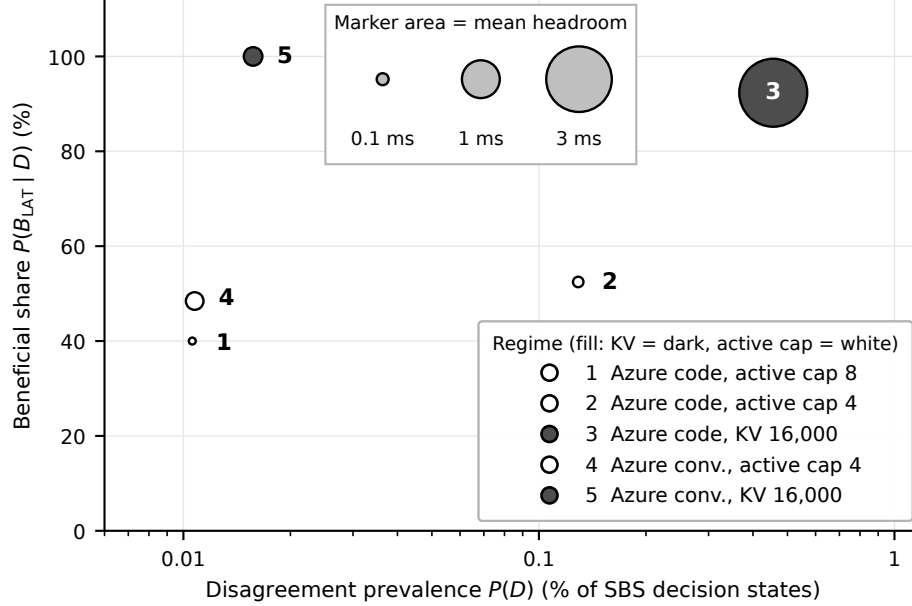


Figure 5: Regime map for the five selected regimes, numbered as in Figure 4 and Table 3. Horizontal axis: disagreement prevalence $P(D)$, the percentage of SBS decision states that are disagreement states (logarithmic scale). Vertical axis: beneficial share $P(B_{LAT} | D)$. Marker area is proportional to the regime’s mean oracle headroom (key in the plot), and fill denotes the pressure axis. $P(B_{LAT} | D)$ counts any positive headroom: regime 5, at 100%, has a mean headroom of 0.24 ms and a maximum below 0.48 ms.

mand and answers “what is the expected headroom at a disagreement state drawn from this sample”; the equal-window and equal-regime means answer the same question for a typical window or regime, and they are not interchangeable with it.

Leaving one unit out gives the same picture. All 36 single-window and 5 single-regime omissions leave a positive mean, and every corresponding bootstrap interval excludes zero. Omitting Azure-code window w11 leaves 0.365 ms (95% CI [0.158, 0.682] ms); omitting the Azure-code KV-capacity = 16,000 regime leaves 0.159 ms (95% CI [0.058, 0.234] ms). Omitting any other window changes the mean only to between 2.00 and 2.26 ms, and omitting any other regime to between 2.02 and 2.39 ms. The sign therefore survives all 41 omissions, whereas the headline magnitude does not survive the omission of the two dominant units.

Table 4: Post hoc practical-effect thresholds: disagreement states whose oracle headroom H_{LAT} exceeds each threshold. The first row is the preregistered criterion; the second excludes one floating-point-residue state with a 10^{-12} s guard. “Share” is of the 720 states; “Windows” counts the windows (of 36) that contain at least one such state; “In KV 16,000” counts states from the Azure-code KV-capacity = 16,000 regime, which contributes 431 of the 720 states.

Criterion	States	Share	Windows	In KV 16,000
Preregistered (> 0 , strict)	590	81.9%	32	398
Guarded ($> 10^{-12}$ s)	589	81.8%	32	398
> 0.1 ms	432	60.0%	17	327
> 0.25 ms	322	44.7%	16	259
> 0.5 ms	212	29.4%	9	201
> 1 ms	188	26.1%	7	185
> 2 ms	162	22.5%	4	161
> 5 ms	129	17.9%	2	128

7.3. Concentration of headroom

Window w11 of the Azure-code trace contains 263 of the 720 states (36.5%) but contributes 88.4% of the total summed headroom; the three largest windows contribute 95.3%, and the effective number of windows by headroom mass is 1.27 (5.7 by state count). At regime level, Azure-code KV-capacity = 16,000 holds 59.9% of the states and 96.8% of the headroom, contributing 1.93 of the 2.00 ms state-weighted mean. Within w11, 205 states of that regime average 6.16 ms. Of the 129 states above 5 ms, 128 are in w11, as are 140 of the 162 states above 2 ms (Figure 6b). The state-weighted mean is thus highly concentrated in a narrow high-headroom subset of states and should not be read as the typical benefit of a disagreement state.

7.4. Uncertainty diagnostics

Monte Carlo error does not explain the width of the preregistered interval. With 10^6 replicates the percentile interval is [0.184, 3.543] ms, no replicate has a nonpositive mean, and the lower endpoint of the 2,000-replicate procedure varies by only 0.004 ms (standard deviation) across 1000 seeds. The BCa interval is [0.239, 4.049] ms. The bootstrap distribution is bimodal: 36% of replicates do not draw window w11 and have a mean of about 0.36 ms, against 2.43 ms for those that do, so the percentile interval spans the gap between

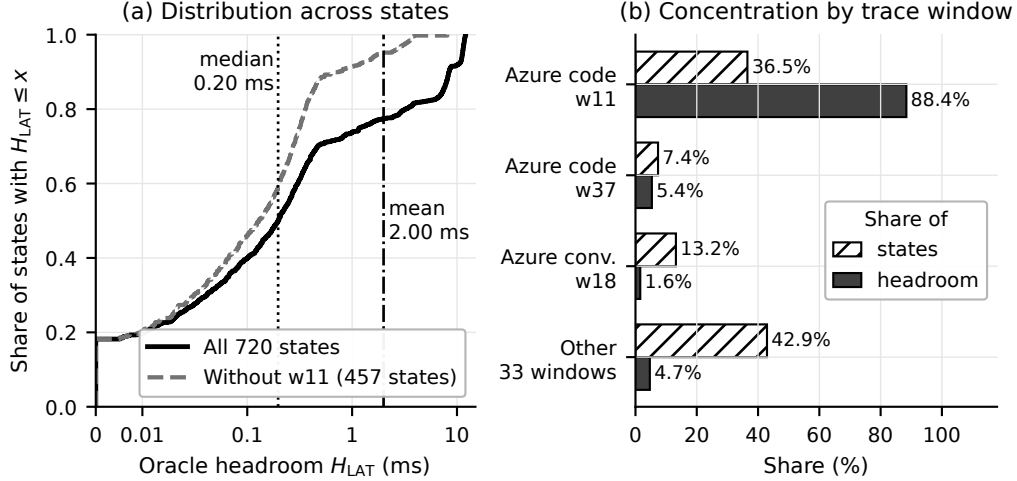


Figure 6: Distribution and concentration of oracle headroom over the 720 disagreement states. (a) Empirical distribution of H_{LAT} with and without Azure-code window w11; zero headroom is the intercept at $x = 0$, and the axis is linear below 0.01 ms and logarithmic above. The dotted and dash-dotted lines mark the median and the state-weighted mean. (b) Share of the 720 states (hatched) and of the total summed headroom (solid) for the three windows with the largest headroom and for the remaining 33 windows.

two populations of replicates. The delete-one-window jackknife t interval, $[-1.36, 5.35]$ ms, includes zero because one influential window inflates its standard error (1.65 ms); all 36 delete-one estimates are positive, and we treat the jackknife interval as a diagnostic of this dependence rather than as a competing interval. Uncertainty about the magnitude of the headroom thus reflects its concentration in few windows, not simulation noise.

8. Objective Sensitivity: Arrival-Normalized Weighted Goodput Saturation

The earlier pressure study completed with causal integrity and complete branch coverage, but all terminal ANWG values were 1.0 and all ANWG advantages were zero. An identifiability audit found no implementation bug: requests completed within deadlines and the objective saturated. Simple sensitivity cases confirmed that ANWG distinguishes missed deadlines or unfinished work, but not different latency or queueing when all requests still meet a loose deadline.

Table 5: Post hoc sensitivity of the mean oracle headroom to the aggregation rule and to leaving out the two dominant units. In the first block each workload, regime, or window receives equal weight, except in the state-weighted row, where each state does. “Share” is the beneficial share under the strict > 0 criterion. The state-weighted row is the pre-registered estimate and interval (2,000 replicates); all other intervals are window-clustered percentile bootstrap intervals with 200,000 replicates (equal-regime and equal-workload treat regimes and workloads as fixed strata; Section 3.4).

Estimand	Units	Mean (ms)	95% CI (ms)	Share
State-weighted (preregistered)	720 states	1.996	[0.191, 3.546]	0.819
Equal-workload	2 workloads	1.360	[0.184, 2.094]	0.809
Equal-regime	5 regimes	0.759	[0.145, 1.161]	0.666
Equal-window	36 windows	0.327	[0.114, 0.644]	0.632
<i>Leave one out: state-weighted mean of the remaining states</i>				
Omit Azure-code window w11	457 states	0.365	[0.158, 0.682]	0.821
Omit Azure-code KV 16,000	289 states	0.159	[0.058, 0.234]	0.664

Most counterfactual branches nevertheless changed latency. Thus zero ANWG headroom does not imply zero latency headroom. We do not use the earlier post hoc latency analysis as confirmatory evidence. The fresh untouched study in Section 6 is the confirmatory latency result, with its own pre-frozen primary endpoint and clustered bootstrap.

This is objective sensitivity, not evidence that ANWG is invalid. It is an appropriate deadline-goodput objective for one question; mean latency is a different operational objective. A serving paper should not promote whichever metric yields the more favorable conclusion.

9. Results: Bounded vLLM System Correspondence Probe

The probe used one RTX 5060 Ti GPU, vLLM 0.27.1, Qwen2.5-0.5B-Instruct, and deterministic token-shape replay. It comprised 40 full-versus-chunked runs and a 20-run native chunked-prefill budget probe, with queueing and scheduler traces retained. It separates three levels of evidence. Level 1, one-step headroom at disagreement states, is established only in the controlled replay. Level 2, that resource pressure changes observable queueing and scheduling behavior, is supported in vLLM. Level 3, that a deployed adaptive selector would recover the simulated headroom or improve end-to-

Table 6: Correspondence between the simulator and the vLLM probe. The vLLM column reflects pressure and action correspondence; it does not estimate one-step causal headroom.

Property	Simulator/replay	Real vLLM
Low pressure	Canonical collapse	Low queue baseline
Resource binding	KV/active caps induce D	Queue and active-sequence pressure observed
Action difference	SBS versus six-policy canonical action	Native token-budget trace signature
Latency consequence	One-step, SBS-relative	Whole-run budget tradeoff
Exact intervention	Available	Not available

end latency, SLOs, or throughput, is not established.

The native budget probe held chunked prefill fixed and varied the per-step token budget (512 tokens, T512, versus 4,096 tokens, T4096). T512 produced 1,680 scheduler steps, 165 mixed prefill/decode steps, and 194 partial-prefill items; T4096 produced 1,536, 55, and 9. Waiting queues reached lengths of 7 and 5, respectively, while running sequences reached 4 in both cases. The low-load regime showed a 30.6 ms difference in late time to first token and a 16.3 ms prompt-heavy end-to-end tradeoff; the high-load regime showed a smaller time-to-first-token difference and a 23.3 ms prompt-heavy end-to-end tradeoff.

The direct full-versus-chunked comparison did not reproduce the simulator’s qualitative reversal (Table 6). KV utilization and preemption metrics were unavailable in the native probe because its metric parser returned empty dictionaries. The probe therefore shows that concrete serving pressure alters observable scheduling behavior and latency; it does not estimate Eq. 3 in the real system.

10. Discussion

10.1. Sign and magnitude of the headroom

The fresh study establishes that positive one-step headroom exists: under the preregistered criterion, forcing an alternative action lowers mean latency

in 590 of 720 disagreement states (81.9%). The sign is stable across the aggregations and leave-one-out analyses of Section 7. The magnitude is not. The preregistered state-weighted mean of 2.00 ms (95% window-clustered CI [0.19, 3.55] ms) estimates the expected headroom at a state drawn from this sample, not the headroom of a typical state: the median is 0.20 ms, and the mean falls to 1.36, 0.76, and 0.33 ms when workloads, regimes, or windows receive equal weight (Table 5). “Beneficial” is a criterion on the sign of the causal effect, not on its operational size: 29.4% of disagreement states offer more than 0.5 ms and 22.5% more than 2 ms (Table 4).

The mean reflects where the headroom lies. One Azure-code window carries 88.4% of the total headroom, the Azure-code KV-capacity = 16,000 regime carries 96.8%, and the effective number of windows by headroom mass is 1.27 (Figure 6). Without that window the mean is 0.36 ms (95% CI [0.16, 0.68] ms); without that regime it is 0.16 ms (95% CI [0.06, 0.23] ms). The window is part of the valid fresh sample. It shows that an aggregate mean can conceal highly localized scheduling opportunity, so the distribution of effects, the prevalence of material effects, and the sensitivity to aggregation belong next to any overall mean.

The interval diagnostics agree. A 10^6 -replicate percentile interval ([0.18, 3.54] ms) matches the preregistered one, and the BCa interval ([0.24, 4.05] ms) stays positive, so Monte Carlo error does not limit the inference. The delete-one-window jackknife interval includes zero, and the bootstrap distribution is bimodal according to whether window w11 is drawn. Both reflect dependence on one cluster: with 36 windows but only about one effective window carrying the headroom, the preregistered interval is wide because cluster diversity is limited. It remains the pre-specified inference; the diagnostics locate the source of its width.

10.2. Opportunity, prediction, and deployment

Adaptive scheduling should be evaluated in stages, because each stage can fail while the previous one holds. Resource pressure need not produce executable disagreement: native replay met no alternative action in about one million decision states. Disagreement need not be beneficial, a positive effect need not be material, and a material oracle effect need not be recoverable. Collapsing the stages overstates opportunity at every step: a virtual-best-solver gap or complementarity in policy scores does not show that the live system reaches states with a different executable action, and an oracle mean

of about 2 ms reads as the value of adaptation although most disagreement states offer a fraction of it.

The estimand is one-step oracle headroom under SBS continuation: one alternative action is forced at a disagreement state, and the default then resumes. $H_{\text{LAT}}(s)$ selects the best alternative after the outcomes are observed, so it answers whether a useful action existed at s , not whether an online system could have identified it, and it does not measure the value of a fully adaptive policy, of longer-horizon switching, of a selector, or of adaptation overhead. Those are the later stages of Eq. 1; separating opportunity from predictability lets a weak or absent opportunity rule out selector effort before it is spent.

10.3. Resource pressure and action support

Arrival-rate scaling up to eight times left every workload family action-null, whereas active-sequence and KV-capacity caps created disagreement in all three (Section 5). In these production-derived workloads under controlled resource interventions, opportunity is tied to particular forms of resource binding rather than to load as a scalar. Workload intensity and resource contention are different quantities: a system can be busy without facing a scheduling choice, and can face one at the native arrival rate once a capacity constraint binds. The relevant operating region is defined by which resource binds, not by arrival rate alone.

10.4. System correspondence

The vLLM experiment is a bounded system correspondence probe (Section 9). It shows that resource pressure changes observable queueing and scheduling behavior in a real engine: a smaller chunked-prefill token budget produced more mixed prefill-decode steps and more partial-prefill items than a larger one. It did not reproduce the simulator’s qualitative reversal in the direct full-versus-chunked comparison, it has no exact one-step intervention, and it does not validate the simulated headroom magnitude or an adaptive-scheduler gain. The probe supports the qualitative link between resource pressure and scheduling behavior; the magnitude question is answered only by the simulated counterfactuals.

10.5. Implications for practice

For a system designer, the evidence supports five practices.

- Measure executable action disagreement before building a selector; diversity in policy scores is not evidence of it.
- Characterize which resource-binding regimes the system enters, rather than only its arrival rate.
- Report the distribution of effect sizes, the prevalence of the states where they occur, and their concentration, alongside the oracle mean.
- Weigh the recoverable gain against selector, monitoring, and switching costs; the median disagreement state offers only 0.20 ms.
- Outside the few regimes with material opportunity, staying with a fixed scheduler may be the appropriate policy.

10.6. Relation to modern adaptive systems

LMetric, SOLA, Llumnix, MorphServe, Strata, Libra, and FastServe address how to adapt serving behavior under dynamic resource and SLO conditions; ServeGen, BurstGPT, the Azure traces, OpenTela, and longitudinal trace studies address how to obtain and operate on realistic workload evidence. The staged measurement here is complementary. It identifies the operating regions in which such mechanisms have an executable alternative to exploit, and it quantifies the headroom that a strong default leaves there.

10.7. Limitations and threats to validity

The limitations below bound what the evidence supports.

Policy portfolio. The six policies are not the universe of schedulers, and other portfolios may expose different action support. The results are default-relative measurements for this portfolio.

Workloads. Three production-derived families were studied. Native replay and pressure mapping cover all three, but fresh causal labeling covers selected Azure regimes because BurstGPT contributed no selected regime.

Controlled interventions. Active-sequence and KV caps are experimental interventions, not observed production provisioning. They identify which forms of binding create disagreement, not how often production systems operate under them.

One-step oracle. H_{LAT} isolates one-step causal opportunity with an ex post best alternative; it is not closed-loop adaptation.

Concentration. The magnitude rests on few windows, with an effective number of about 1.27 by headroom mass. The sign has broader support: 32 of the 36 windows contain a state with positive headroom.

System correspondence. The probe uses one model, one GPU, one vLLM version, token-shape replay, and incomplete instrumentation (KV utilization and preemption metrics were unavailable), so it supports only the qualitative mechanism.

Objective. Mean latency is the primary causal endpoint. The goodput audit shows that the objective changes the measured opportunity (Section 8); tail-latency and SLO-violation objectives were not evaluated.

External validity. Other models, hardware, schedulers, context-length distributions, batching systems, and resource managers may show a different opportunity structure. The staged protocol transfers; the measured magnitudes need not.

Directions that follow from the decomposition are to learn a conservative selector only in regimes where measured opportunity is material, to test whether those regimes recur on other hardware and models, and to extend the counterfactual to longer horizons and tail-latency objectives with explicit accounting of adaptation overhead.

11. Reproducibility and Artifact Scope

The public artifact described under Data and Code Availability records trace provenance, frozen manifests, support grids, fresh non-overlap proofs, causal state and action manifests, source hashes, bootstrap seed 20260920, and the vLLM environment. It covers the four result groups reported here: native replay, pressure mapping, fresh confirmatory causal latency, and the bounded vLLM system correspondence probe. Artifacts from earlier selector studies remain in the repository but are not treated as evidence of learned deployment readiness.

12. Conclusion

This paper asked when adaptive scheduling has anything to act on in LLM serving, before any adaptive scheduler is built. Replay of production-derived traces under abundant resources offered essentially nothing: none of about one million decision states had an executable alternative to the single best scheduler (SBS). Controlled active-sequence and KV-capacity caps, but not arrival-rate scaling up to eight times, created such states, and fresh one-step interventions showed that forcing an alternative there lowers latency in most of them (590 of 720 under the preregistered criterion). A bounded vLLM probe supports the qualitative link between pressure and scheduling behavior without validating the simulated magnitudes.

The headroom is positive but small and concentrated. The state-weighted mean of 2.00 ms is not a typical effect: the median disagreement state offers 0.20 ms, the mean falls to 0.33 ms when windows are weighted equally, and one trace window holds 88.4% of the total. Adaptation opportunity is therefore real, sparse, pressure-dependent, and generally modest, and its existence does not imply deployment value.

The broader lesson is that scheduler adaptation should be justified stage by stage, from resource pressure through executable disagreement and causal benefit to the distribution and prevalence of the effect, before selector complexity is added. Reporting concentration and aggregation sensitivity, and not only a mean, distinguishes a localized opportunity from a general one. Adaptation is justified by where the measured opportunity lies, not by its average.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the author used ChatGPT (OpenAI), Codex (OpenAI), Gemini (Google), and Claude (Anthropic) in order to assist with organization, language editing, literature-search planning, and drafting analysis, figure, and test code. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

Declaration of Competing Interest

The author declares that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The author gratefully acknowledges Professor Ioannis Koutis for research guidance and scientific feedback. The author thanks the NJIT Wulver HPC for providing computational resources for the fresh confirmatory computation, and Anders Borum (Secure ShellFish) for providing tools that enabled remote research access. Finally, the author thanks his mother for her continued support.

Funding and Support

This work received in-kind computational support through the Google Cloud Research Credits Program (USD 1,000 in cloud credits) and computational/tooling support from CloudRift Inc. Neither Google nor CloudRift Inc. had any role in the study design, data collection, analysis, interpretation, manuscript preparation decisions, or the decision to submit the article for publication. No monetary research grant was received from these organizations.

Data and Code Availability

Code, derived research artifacts, experiment configurations, and reproducibility materials for this study are available in the GitHub repository <https://github.com/SoroushVahidi/llm-serving-heuristic-evolution> (release **performance-evaluation-v1.1.0**) and are permanently archived on Zenodo [28]. The archive is a snapshot of that release: the published v1.0.0 record predates the post hoc robustness analysis, and the materials it lacks (the robustness outputs, the corrected derivative, the continuation shards, the figure code, and the claim manifest) are available in the GitHub repository only until the v1.1.0 version of the archive is published. The v1.1.0 archive contains the frozen confirmatory artifacts (native replay, pressure mapping, and fresh causal latency headroom), the post hoc

robustness outputs (Section 7), a corrected derivative of two descriptive state-level columns that leaves the original frozen file unchanged, the continuation shards from which that derivative is computed, the simulator and analysis code, the figure and table scripts, and the claim manifest that checks the quoted numbers. Raw third-party workload traces are not redistributed and are obtained from their original upstream sources.

References

- [1] J. R. Rice, The algorithm selection problem, *Advances in Computers* 15 (1976) 65–118.
- [2] C. P. Gomes, B. Selman, Algorithm portfolios, *Artificial Intelligence* 126 (1–2) (2001) 43–62.
- [3] B. Sun, Z. Huang, H. Zhao, W. Xiao, X. Zhang, Y. Li, W. Lin, Llumnix: Dynamic scheduling for large language model serving, in: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), USENIX Association, 2024, pp. 173–191.
- [4] K. Hong, X. Li, L. Chen, Q. Mao, G. Dai, X. Ning, S. Yan, Y. Liang, Y. Wang, SOLA: Optimizing SLO attainment for large language model serving with state-aware scheduling, in: *Proceedings of Machine Learning and Systems*, Vol. 7, 2025.
- [5] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, H. Zhang, DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving, in: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), USENIX Association, 2024, pp. 193–210.
- [6] D. Zhang, J. Han, K. Zhang, X. Wei, S. Shen, C. Fang, W. Yu, J. Zhou, R. Chen, LMetric: Simple is better – multiplication may be all you need for LLM request scheduling, in: 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26), USENIX Association, 2026.
- [7] T. Choudhury, G. Joshi, W. Wang, Job assignment in machine learning inference systems with accuracy constraints, *Performance Evaluation* 167 (2025) 102463. doi:10.1016/j.peva.2024.102463.

- [8] M. Yildiz, A. Rolich, A. Baiocchi, Dispatching policies in data center clusters: Insights from Google and Alibaba workloads, *Performance Evaluation* 172 (2026) 102551. doi:10.1016/j.peva.2026.102551.
- [9] Q. Liang, H. Wang, P. Shenoy, Lilou: Resource-aware model-driven latency prediction for GPU-accelerated model serving, *Performance Evaluation* 171 (2026) 102539. doi:10.1016/j.peva.2025.102539.
- [10] A. Ali, X. Ma, S. Zawad, P. Aditya, I. E. Akkus, R. Chen, L. Yang, F. Yan, Enabling scalable and adaptive machine learning training via serverless computing on public cloud, *Performance Evaluation* 167 (2025) 102451. doi:10.1016/j.peva.2024.102451.
- [11] Microsoft Azure, Azure LLM inference trace 2023, <https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2023.md>, public LLM inference trace dataset released by Azure (2023).
- [12] Y. Wang, Y. Chen, Z. Li, X. Kang, Y. Fang, Y. Zhou, Y. Zheng, Z. Tang, X. He, R. Guo, X. Wang, Q. Wang, A. C. Zhou, X. Chu, BurstGPT: A real-world workload dataset to optimize LLM serving systems, in: *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ACM, 2025, pp. 5831–5841. doi:10.1145/3711896.3737413.
- [13] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, I. Stoica, Efficient memory management for large language model serving with PagedAttention, in: *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP 2023)*, ACM, 2023, pp. 611–626.
- [14] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, B.-G. Chun, Orca: A distributed serving system for transformer-based generative models, in: *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, USENIX Association, 2022, pp. 521–538.
- [15] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, R. Ramjee, Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve, in: *18th USENIX*

Symposium on Operating Systems Design and Implementation (OSDI 24), USENIX Association, 2024, pp. 117–134.

- [16] P. Patel, E. Choukse, C. Zhang, A. Shah, I. Goiri, S. Maleki, R. Bianchini, Splitwise: Efficient generative LLM inference using phase splitting, in: Proceedings of the 51st Annual International Symposium on Computer Architecture, IEEE Computer Society, 2024. doi:10.1109/ISCA59077.2024.00019.
- [17] B. Wu, Y. Zhong, Z. Zhang, S. Liu, F. Liu, Y. Sun, G. Huang, X. Liu, X. Jin, FastServe: Iteration-Level preemptive scheduling for large language model inference, in: 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26), USENIX Association, 2026, pp. 57–74.
- [18] C. Ruan, Y. Chen, D. Tian, Y. Shi, Y. Wu, J. Li, C. Li, Libra: Flexible request partitioning and scheduling for serving unbalanced and dynamic LLM workloads, in: 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26), USENIX Association, 2026.
- [19] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, X. Xu, Mooncake: Trading more storage for less computation – a KVCache-centric architecture for serving LLM chatbot, in: 23rd USENIX Conference on File and Storage Technologies (FAST 25), USENIX Association, 2025, pp. 155–170.
- [20] Z. Xie, Z. Xu, M. Zhao, Y. An, V. S. Mailthody, S. Mahlke, M. Garland, C. Kozyrakis, Strata: Hierarchical context caching for long context language model serving, arXiv preprint arXiv:2508.18572 (2025).
- [21] Z. Su, T. Lan, Z. Wang, J. Yang, Y. Cheng, Efficient and workload-aware LLM serving via runtime layer swapping and KV cache resizing, arXiv preprint arXiv:2506.02006 (2025).
- [22] J. Stojkovic, C. Zhang, I. Goiri, J. Torrellas, E. Choukse, DynamoLLM: Designing LLM inference clusters for performance and energy efficiency, in: Proceedings of the 2025 IEEE International

Symposium on High Performance Computer Architecture (HPCA 25), IEEE, 2025, pp. 1348–1362. doi:10.1109/HPCA61900.2025.00102.

- [23] W. Nixon, J. Durbin, F. Standhartinger, H. S. Gunawi, J. Yang, A year in LLM serving: Workload evolution, caching and load-balancing, arXiv preprint arXiv:2608.13573 (2026).
- [24] Y. Xiang, X. Li, K. Qian, W. Yu, E. Zhai, X. Jin, ServeGen: Workload characterization and generation of large language model serving in production, in: 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26), USENIX Association, 2026.
- [25] X. Yao, Y. Jiang, I. Badanin, Q. Hu, R. M. Smith, B. Yuan, I. Schlag, E. Yoneki, A. Klimovic, OpenTela: Unifying decentralized computing resources for heterogeneous LLM serving, in: 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26), USENIX Association, 2026, pp. 1821–1838.
- [26] L. Xu, F. Hutter, H. H. Hoos, K. Leyton-Brown, SATzilla: Portfolio-based algorithm selection for SAT, *Journal of Artificial Intelligence Research* 32 (2008) 565–606.
- [27] M. Lindauer, H. H. Hoos, F. Hutter, T. Schaub, AutoFolio: An automatically configured algorithm selector, *Journal of Artificial Intelligence Research* 53 (2015) 745–778.
- [28] [dataset] S. Vahidi, Reproducibility package for “When Does LLM-Serving Scheduler Adaptation Matter? Action Opportunity and Causal Headroom in Production-Derived Replay”: data, code and manuscript sources, Zenodo dataset and software archive, version 1.1.0; published v1.0.0 record: <https://doi.org/10.5281/zenodo.22865294>; repository <https://github.com/SoroushVahidi/llm-serving-heuristic-evolution> (2026).